

ELECTROGO APPLICATION DEPLOYMENT ON MICROSOFT AZURE USING TERRAFORM

Automated 2-Tier Deployment of ASP.NET Core Web Application and Azure Database for MySQL

1. Aim of the Project

The aim of this project is to automate the deployment of the **ElectroGO** web application on **Microsoft Azure** using **Terraform** by implementing a **2-tier architecture** with modular Infrastructure as Code. The project provisions the required Azure infrastructure, deploys the ElectroGO ASP.NET Core application on a Linux virtual machine, provisions Azure Database for MySQL as the backend database tier, configures networking and load balancing, and automatically installs and deploys the application during VM provisioning.

The project also uses **Terraform remote state management** by storing the Terraform state file in an **Azure Storage Account Blob Container**, enabling centralized infrastructure state tracking and safer Terraform execution.

2. Project Title

ElectroGO Application Deployment on Microsoft Azure Using Terraform

2-Tier Infrastructure and Automated Application Deployment using ASP.NET Core (.NET) and Azure Database for MySQL

3. Project Overview

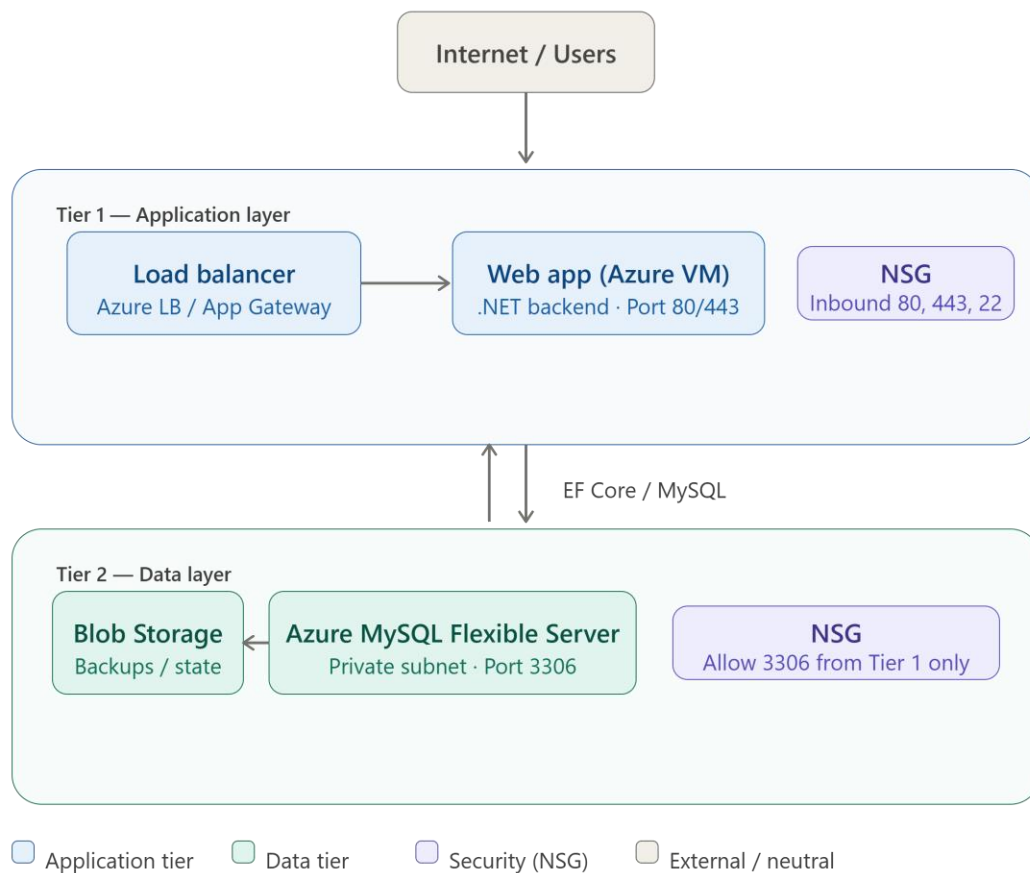
ElectroGO is an EV Charging Station Management Web Application developed using **ASP.NET Core (.NET)**. The objective of this project is to deploy the application in Microsoft Azure using a **2-tier architecture** and **Terraform automation**.

The deployment is implemented using a **modular Terraform repository**, where separate modules are used to provision the **network layer**, **compute layer**, **database layer**, and **load balancer layer**. The ElectroGO application is hosted on an **Azure Linux Virtual Machine**, and the backend database is deployed using **Azure Database for MySQL Flexible Server**.

During deployment, Terraform provisions the Azure resources and executes a **bootstrap shell template** (`electrogo-bootstrap.sh.tftpl`) to prepare the Linux VM. The bootstrap process installs **.NET 8**, **Nginx**, and other required packages, pulls the ElectroGO application source code, configures the database connection dynamically using the generated MySQL endpoint, and starts the application on the server.

The Terraform project also uses a **remote backend** in Azure Storage Account to store the **Terraform state file**, ensuring centralized state management. The Storage Account and container used for this backend were created manually before Terraform initialization.

High-Level 2-Tier Architecture Diagram



4. Objectives of the Project

1. The objectives of this project are:
2. To deploy the **ElectroGO ASP.NET Core web application** on Microsoft Azure.
3. To automate Azure infrastructure provisioning using **Terraform**.
4. To implement a **2-tier architecture** consisting of:
 - a. **Application Tier** – Linux VM hosting the ElectroGO web app
 - b. **Database Tier** – Azure Database for MySQL Flexible Server
5. To structure Terraform code using reusable **modules** for networking, compute, database, and load balancer resources.
6. To provision networking resources such as **VNet, subnets, NSGs, and public IPs**.
7. To provision a **Linux Virtual Machine** for application hosting.
8. To provision an **Azure Load Balancer** to expose or route application traffic.
9. To provision **Azure Database for MySQL Flexible Server** as the application database.

10. To automate application deployment using a **bootstrap shell template** during VM provisioning.
11. To install required software such as **.NET 8, Nginx**, and supporting Linux packages automatically.
12. To dynamically inject the generated **MySQL FQDN / database endpoint** into the application configuration.
13. To store Terraform state remotely in **Azure Blob Storage** for centralized state management.

5. Tech Stack

The following technologies and services are used in this project:

Category	Technology / Service
Application Name	ElectroGO
Application Framework	ASP.NET Core (.NET 8)
Cloud Platform	Microsoft Azure
Infrastructure as Code	Terraform
Operating System	Ubuntu Linux Virtual Machine
Database	Azure Database for MySQL Flexible Server
Web Server / Reverse Proxy	Nginx
Load Balancing	Azure Load Balancer
Version Control	Git / GitHub
State Management	Azure Storage Account Blob Backend
Provisioning Script	Bash bootstrap template (electrogo-bootstrap.sh.tftpl)

6. Prerequisites

- Before running the deployment, the following prerequisites were required:
 - **6.1 Azure Subscription and Permissions**
 - A valid Microsoft Azure subscription
 - Permission to create networking, compute, database, and storage resources
 - **6.2 Local Development / Execution Environment**
 - Terraform installed

- Azure CLI installed
- Git installed
- access to the ElectroGO Terraform repository

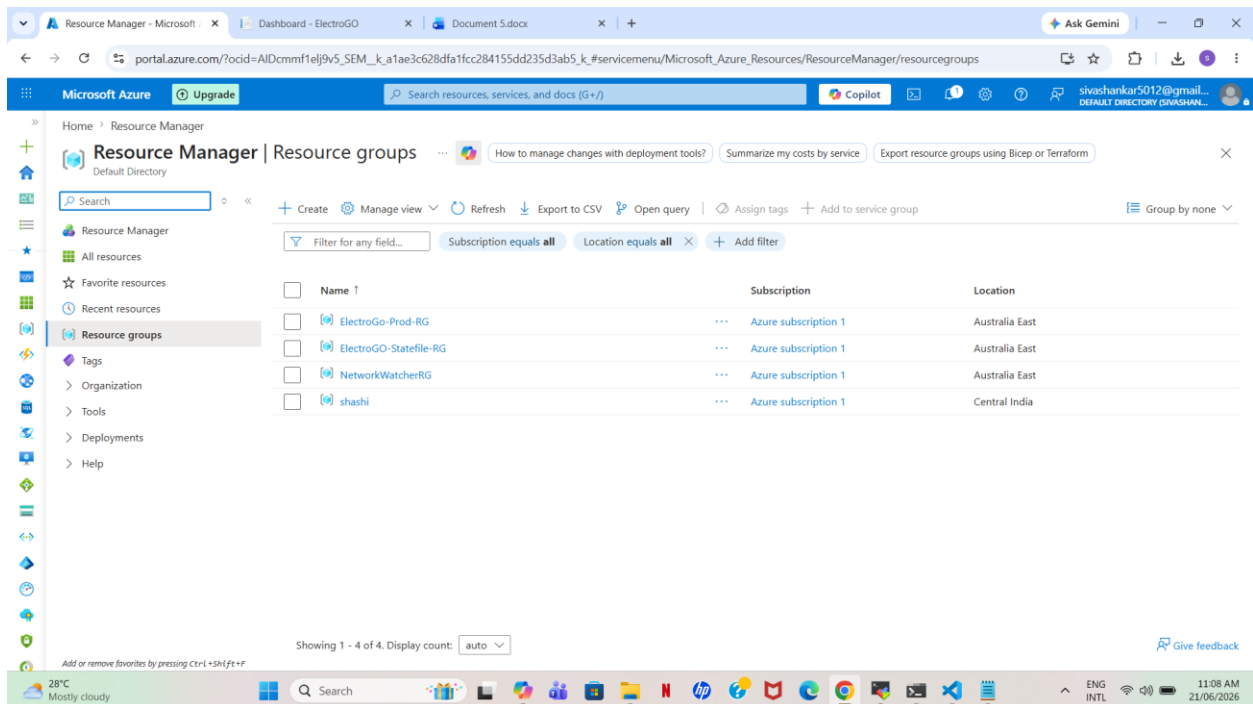
• 6.3 Application Source Code

- ElectroGO ASP.NET Core application code available in GitHub / Git repository
- application compatible with MySQL connectivity

• 6.4 Terraform Remote State Backend

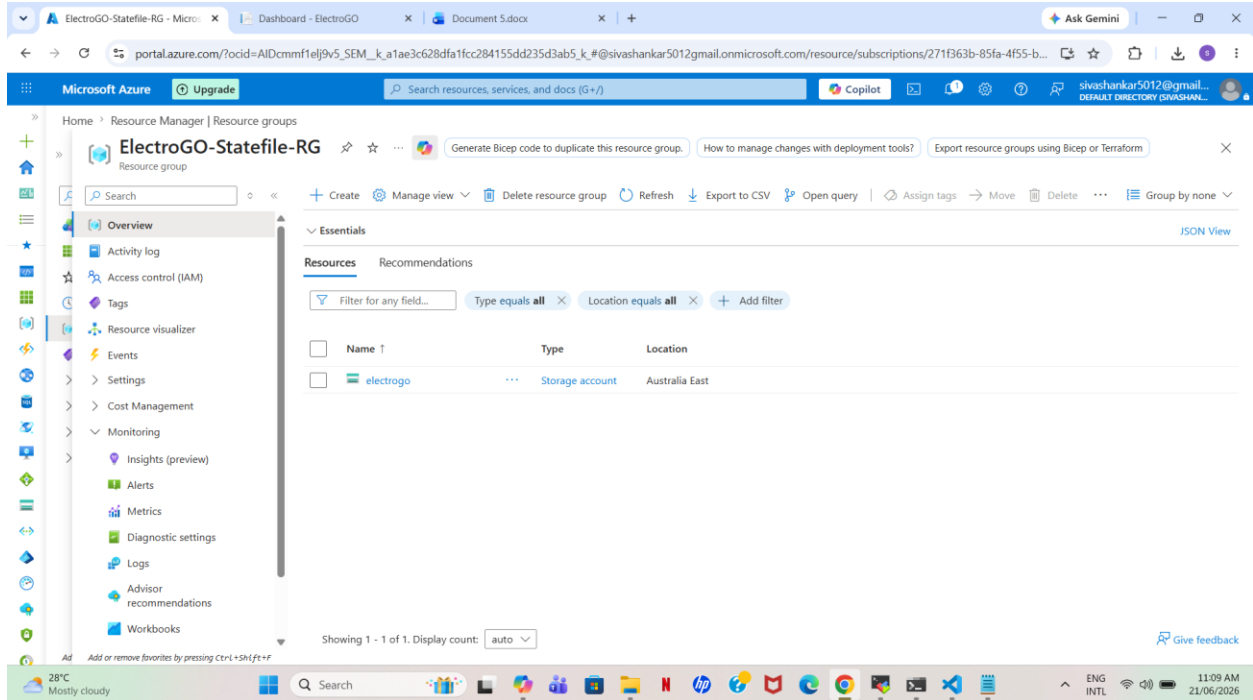
- Before running Terraform, the **Azure Storage Account** and **Blob Container** used to store the Terraform state file were created manually.
- This backend is required because Terraform must have a pre-existing location to store the state file before initialization. The remote backend ensures centralized and persistent storage of the Terraform state file.

Azure Storage Account used for Terraform remote backend

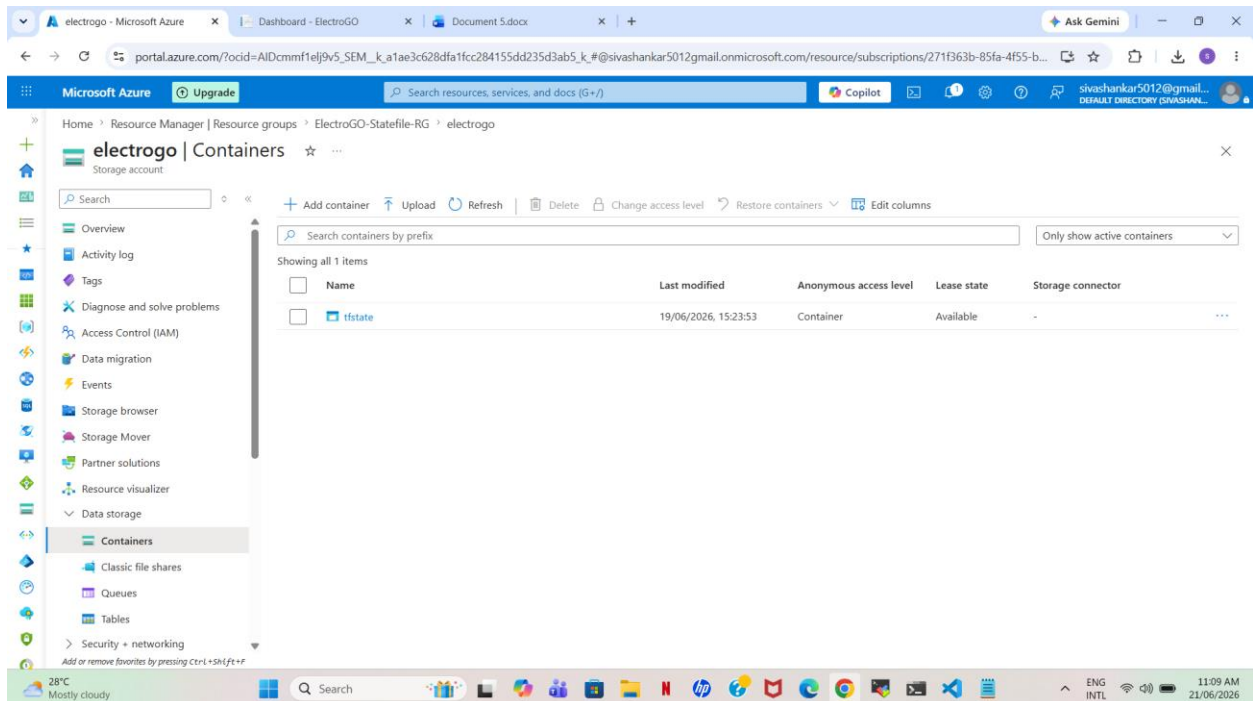


The screenshot shows the Microsoft Azure portal interface. The left sidebar contains navigation options like Home, Resource Manager, All resources, Favorite resources, Recent resources, and Resource groups. The main area displays a table of resource groups under the 'Resource groups' section. The table has columns for Name, Subscription, and Location. Four resource groups are listed: ElectroGo-Prod-RG, ElectroGO-Statefile-RG, NetworkWatcherRG, and shashi, all associated with 'Azure subscription 1' and located in 'Australia East' or 'Central India'.

Name	Subscription	Location
ElectroGo-Prod-RG	Azure subscription 1	Australia East
ElectroGO-Statefile-RG	Azure subscription 1	Australia East
NetworkWatcherRG	Azure subscription 1	Australia East
shashi	Azure subscription 1	Central India



Blob Container storing terraform.tfstate



7. Project Architecture

The ElectroGO deployment follows a **2-tier architecture**, which separates the application layer from the database layer.

- **7. Solution Architecture**

- The ElectroGO deployment follows a **2-tier cloud architecture** with additional supporting infrastructure components.

- **7.1 Application Tier**

- The application tier contains the **Linux Virtual Machine** that hosts the **ElectroGO ASP.NET Core web application**. This VM is provisioned through the **compute module** and configured using the bootstrap template.
- The application tier performs the following:
 - hosts the ElectroGO web application
 - installs .NET runtime and application dependencies
 - runs the application as a service
 - serves traffic through Nginx
 - connects to Azure Database for MySQL

- **7.2 Database Tier**

- The database tier consists of **Azure Database for MySQL Flexible Server**, provisioned through the **database module**. It stores the application's backend data and is accessed by the ElectroGO application using the generated MySQL FQDN.

- **7.3 Supporting Infrastructure**

- The deployment also includes:
 - **Virtual Network**
 - **Subnets**
 - **Network Security Groups**
 - **Public IP**
 - **Network Interface**
 - **Azure Load Balancer**
 - **Azure Storage Account backend for Terraform state**

Azure Resource Group – ElectroGo-Prod-RG

Resource Manager - Microsoft | Dashboard - ElectroGO | Document 5.docx | Ask Gemini

portal.azure.com/?ocid=AIDcmmf1ej9v5_SEM_k_a1ae3c628dfa1fcc284155dd235d3ab5_k_#servicemenu/Microsoft_Azure_ResourceManager/resourcegroups

Microsoft Azure Upgrade Search resources, services, and docs (G+)

Home > Resource Manager

Resource Manager | Resource groups

Default Directory

Search

+ Create Manage view Refresh Export to CSV Open query Assign tags Add to service group Group by none

Filter for any field... Subscription equals all Location equals all Add filter

Name	Subscription	Location
ElectroGo-Prod-RG	Azure subscription 1	Australia East
ElectroGO-Statefile-RG	Azure subscription 1	Australia East
NetworkWatcherRG	Azure subscription 1	Australia East
shashi	Azure subscription 1	Central India

Showing 1 - 4 of 4. Display count: auto

28°C Mostly cloudy

Dashboard - ElectroGO | ElectroGo-Prod-RG - Microsoft | Document 5.docx | 2-Tier Terraform Automation | Ask Gemini

portal.azure.com/?ocid=AIDcmmf1ej9v5_SEM_k_a1ae3c628dfa1fcc284155dd235d3ab5_k_#@sivashankar5012gmail.onmicrosoft.com/resource/subscriptions/271f363b-85fa-4f55-b...

Microsoft Azure Upgrade Search resources, services, and docs (G+)

Home > Resource Manager | Resource groups

ElectroGo-Prod-RG

Resource group

Generate Bicep code to duplicate this resource group. Help me generate Terraform for this resource group configuration. How to manage changes with deployment tools?

Search

+ Create Manage view Delete resource group Refresh Export to CSV Open query Assign tags Move Delete Group by none

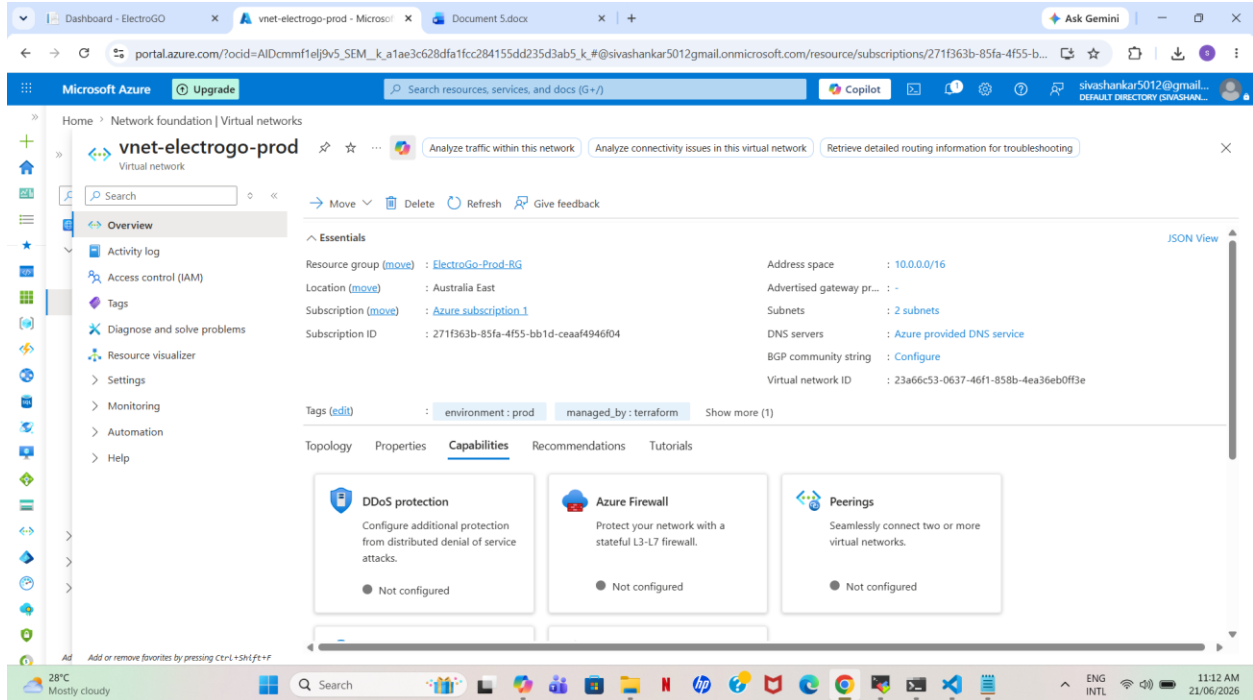
Filter for any field... Type equals all Location equals all Add filter

Name	Type	Location
lb-electrogo-prod	Load balancer	Australia East
mysql-electrogo-prod-kt3wba	Azure Database for MySQL flexible server	Australia East
mysql-electrogo-prod.private.mysql.database.azure.com	Private DNS zone	Global
nic-vm-electrogo-prod	Network Interface	Australia East
nsg-vnet-electrogo-prod-private	Network security group	Australia East
nsg-vnet-electrogo-prod-public	Network security group	Australia East
pip-lb-electrogo-prod	Public IP address	Australia East
pip-vm-electrogo-prod	Public IP address	Australia East
vm-electrogo-prod	Virtual machine	Australia East
vm-electrogo-prod.OsDisk_1_e93aa79431234650b3b1fe78e1dcd527	Disk	Australia East

Showing 1 - 10 of 11. Display count: auto

11:54 AM 21/06/2026

Virtual Network and Subnet Overview



Network Security Group Rules

Dashboard - ElectroGO x ns-g-vnet-electrogo-prod-public x Document 5.docx x + Ask Gemini

portal.azure.com/?ocid=AIDcmf1ej9v5_SEM_k_a1ae3c628dfa1fcc284155dd235d3ab5_k_#@sivashankar5012gmail.onmicrosoft.com/resource/subscriptions/271f363b-85fa-4f55-b...

Microsoft Azure Upgrade Search resources, services, and docs (G+) Copilot sivashankar5012@gmail... DEFAULT DIRECTORY (SIVASHANKAR...)

Home > Network foundation | Network security groups

ns-g-vnet-electrogo-prod-public Network security group

Search Move Delete Refresh Give feedback

Overview

Activity log

Access control (IAM)

Tags

Diagnose and solve problems

Resource visualizer

Settings

Monitoring

Automation

Help

Essentials

Resource group (move): ElectroGo-Prod-RG

Location: Australia East

Subscription (move): Azure subscription 1

Subscription ID: 271f363b-85fa-4f55-bb1d-ceaf4946f04

Tags (edit): environment: prod managed_by: terraform More (1)

Custom security rules: 5 inbound, 0 outbound

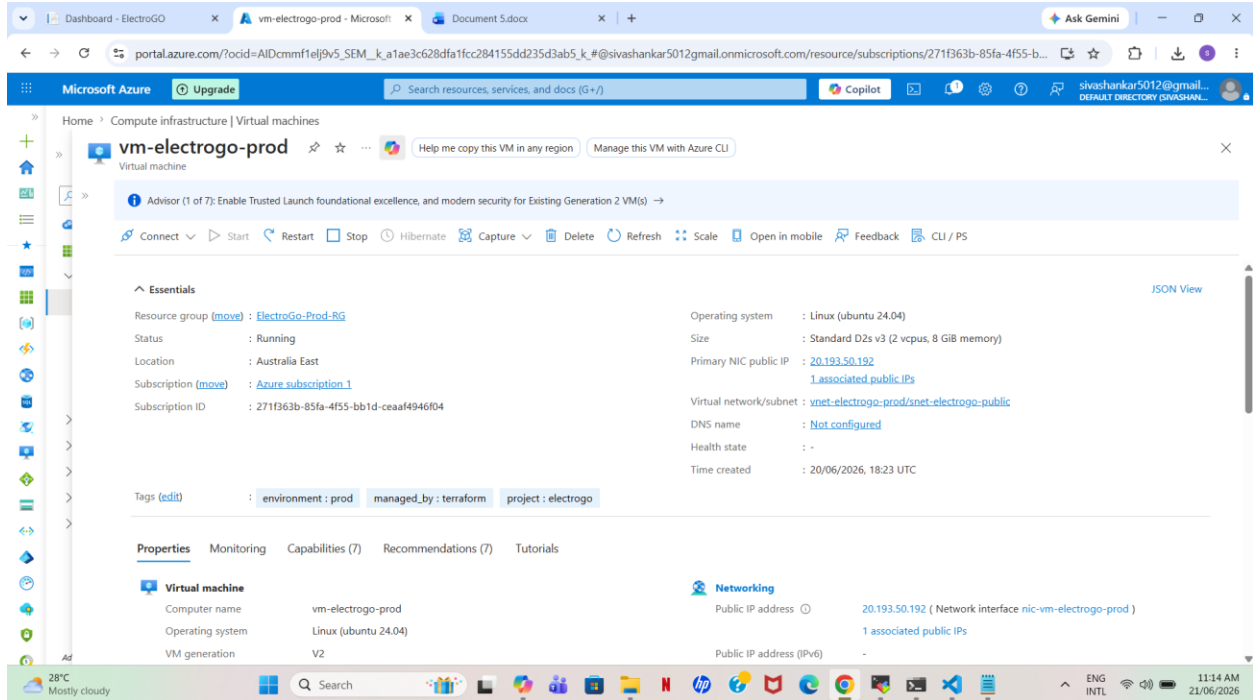
Associated with: 1 subnets, 0 network interfaces

Filter by name Port == all Protocol == all Source == all Destination == all Action == all

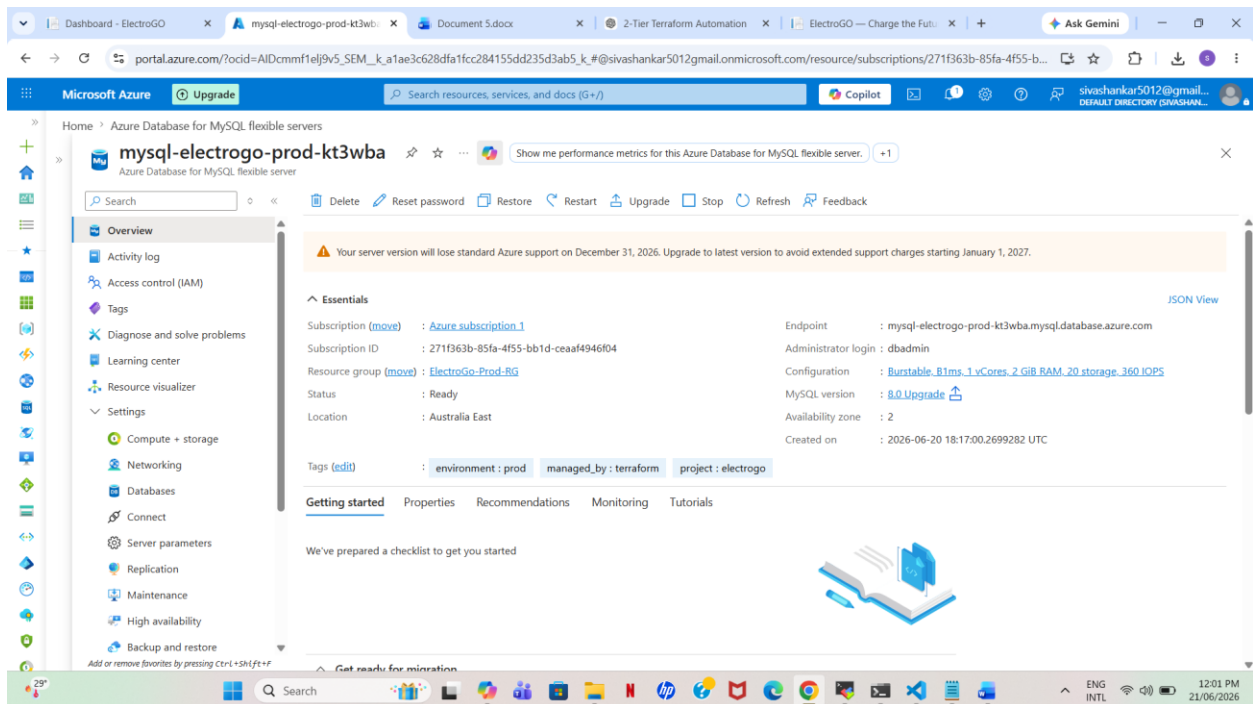
Priority	Name	Port	Protocol	Source	Destination	Action
Inbound Security Rules						
100	Allow-SSH	22	Tcp	Any	Any	Allow
110	Allow-HTTP	80	Tcp	Any	Any	Allow
120	Allow-HTTPS	443	Tcp	Any	Any	Allow
130	ASP-NET	5000	Tcp	Any	Any	Allow
150	sql	3306	Tcp	Any	Any	Allow
65000	AllowVnetInBound	Any	Any	VirtualNetwork	VirtualNetwork	Allow
65001	AllowAzureLoadBalance...	Any	Any	AzureLoadBalancer	Any	Allow
65500	DenyAllInBound	Any	Any	Any	Any	Deny
Outbound Security Rules						
65000	AllowVnetOutBound	Any	Any	VirtualNetwork	VirtualNetwork	Allow
65001	AllowInternetOutBound	Any	Any	Any	Internet	Allow
65500	DenyAllOutBound	Any	Any	Any	Any	Deny

28°C Mostly cloudy Search 11:13 AM 21/06/2026

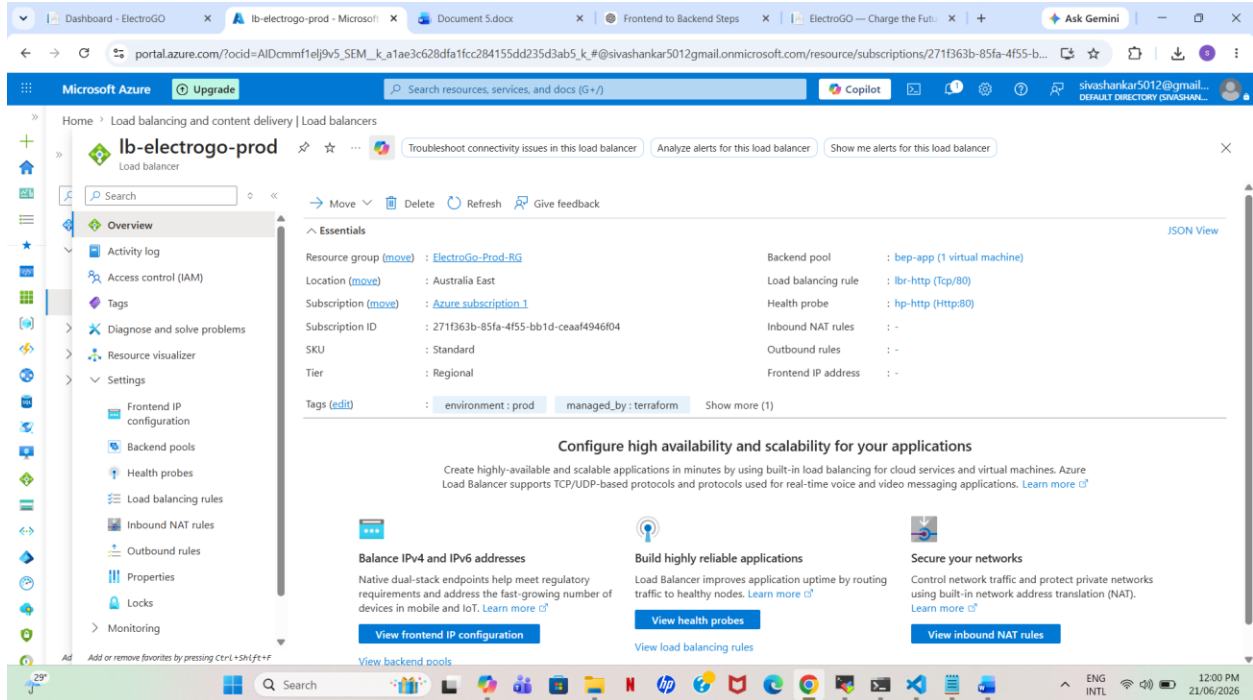
WebApp-Server Virtual Machine Overview



Screenshot 8: Azure Database for MySQL Overview



Azure Load Balancer Overview



8. Azure Resources Used in the Project

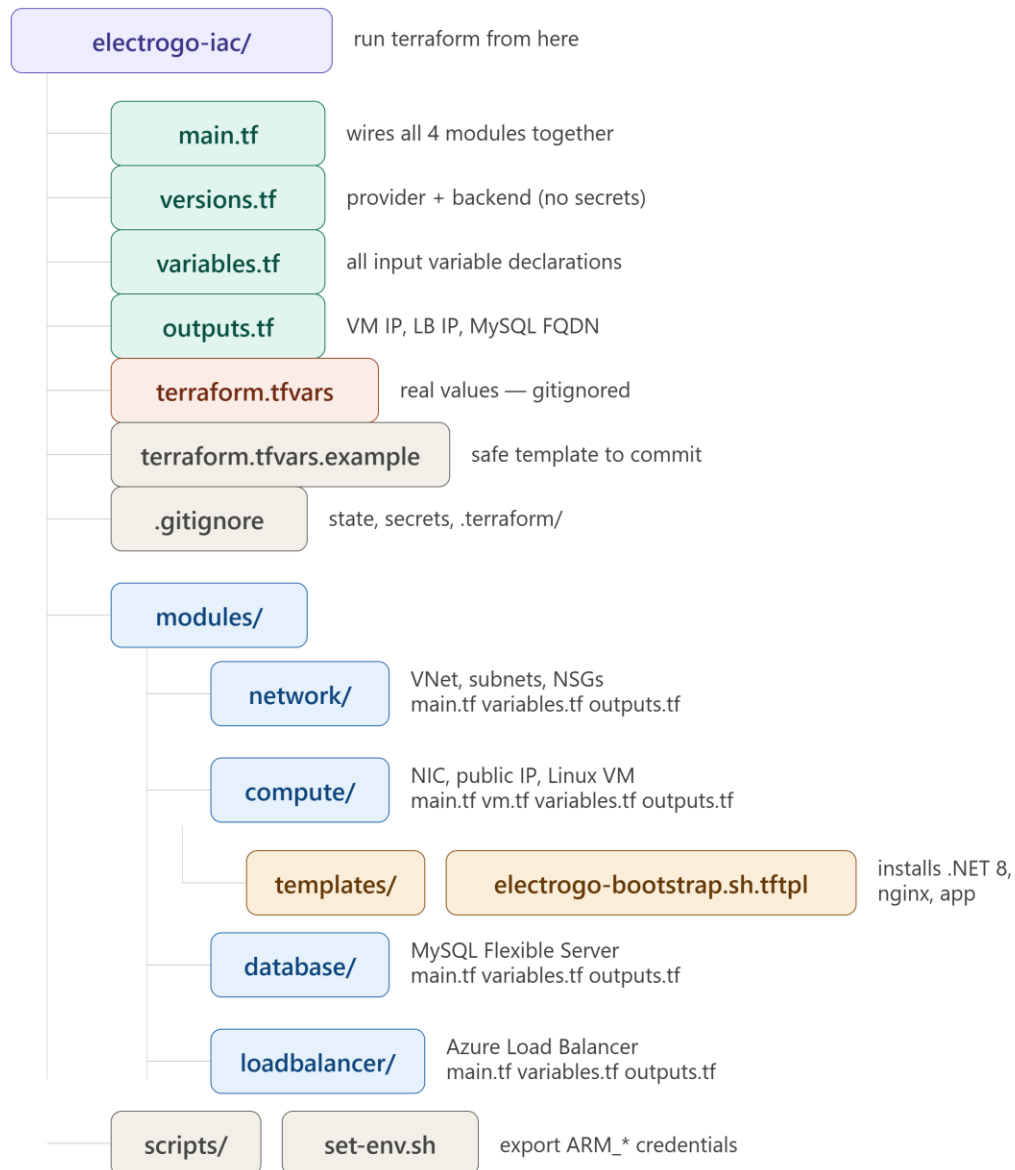
The following Azure resources are used in this project:

Azure Resource	Purpose
Resource Group	Logical container for all project resources
Virtual Network (VNet)	Provides network isolation and communication between resources
Subnet	Segregates network resources within the VNet
Network Security Group (NSG)	Controls inbound and outbound traffic
Public IP	Provides internet access to the web application VM
Network Interface (NIC)	Connects the VM to the virtual network
Linux Virtual Machine	Hosts the ElectroGO web application
Azure Database for MySQL	Database tier for storing application data
Azure Storage Account	Stores Terraform state file remotely
Blob Container	Stores terraform.tfstate within the storage account

9. Terraform Project Structure

The infrastructure and deployment automation are managed using Terraform files. The project contains multiple Terraform configuration files, each responsible for a specific part of the infrastructure.

A typical project structure is as follows:



9. Description of Root Terraform Files

The root-level Terraform files in the `electrogo-iac` repository are responsible for configuring Terraform, defining variables, passing module inputs, and exposing outputs.

9.1 `main.tf`

This file acts as the **entry point of the Terraform project**. It wires together all infrastructure modules:

- `network`
- `compute`
- `database`
- `loadbalancer`

It passes variables between modules and orchestrates the full infrastructure deployment.

9.2 `versions.tf`

This file defines:

- the **Terraform version requirements**
- the **Azure provider configuration**
- the **Terraform backend configuration** for storing the state file in Azure Storage

It contains provider and backend settings, but does not store secrets directly.

9.3 `variables.tf`

This file contains **all input variable declarations** used across the Terraform project. These variables allow the deployment to be configurable without hardcoding values in resource definitions.

Examples include:

- Azure region
- resource names
- VM settings
- MySQL settings
- network values
- application configuration values

9.4 `outputs.tf`

This file exposes useful output values after deployment, such as:

- **VM public IP**
- **Load Balancer IP**
- **MySQL FQDN**

These outputs help validate the deployment and are also useful for application configuration and access.

9.5 terraform.tfvars

This file stores the **actual deployment values** for the Terraform variables. Since it contains environment-specific values, it is typically excluded from version control.

9.6 terraform.tfvars.example

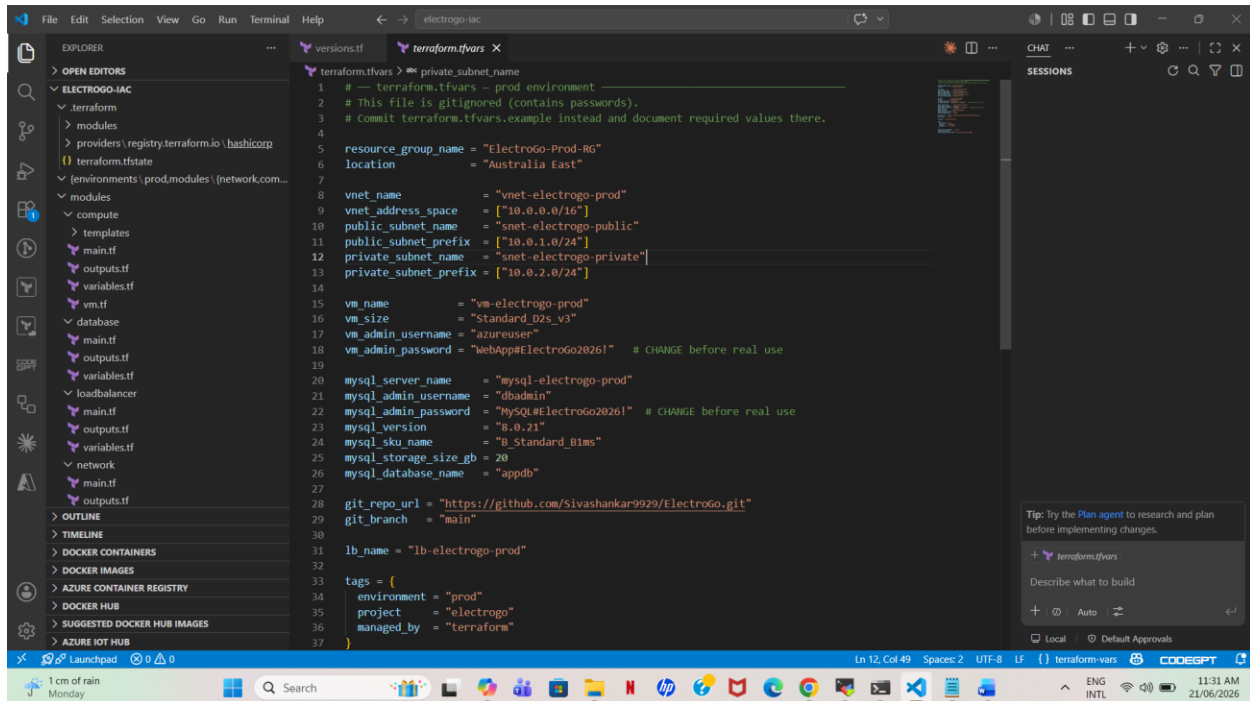
This file acts as a **safe template** for the variable file. It can be committed to the repository and used as a reference for creating the real terraform.tfvars.

9.7 .gitignore

This file prevents sensitive or generated files from being committed to the repository, such as:

- .terraform/
- Terraform state files
- secrets or environment-specific values

Terraform project folder structure in VS Code



10. Terraform Modules Used in the Project

The project uses a **modular Terraform design** to improve readability, maintainability, and reusability. Each module is responsible for a separate infrastructure layer.

11. Network Module

The **network module** provisions the networking resources required by the ElectroGO deployment.

Resources handled by the network module

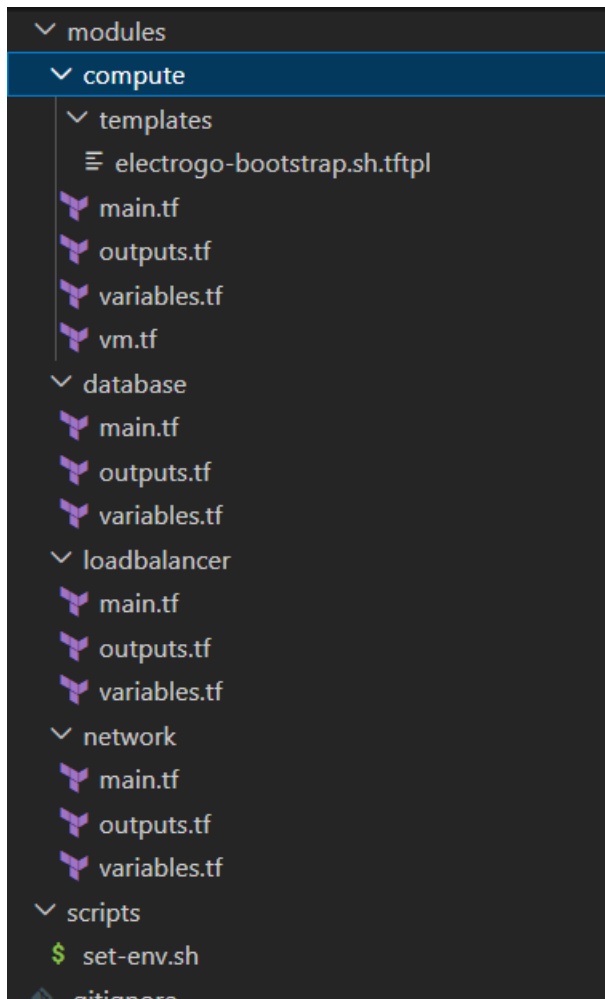
- **Virtual Network (VNet)**
- **Subnets**
- **Network Security Groups (NSGs)**

The network module contains:

- **main.tf**
- **variables.tf**
- **outputs.tf**

This module ensures that the compute resources and database resources are deployed in a secure and organized network environment.

modules/network folder structure



Azure VNet and subnet resources created by the network module

Microsoft Azure | Network foundation | Virtual networks

Search resources, services, and docs (G+)

Filter for any field... Subscription equals all Resource Group equals all Location equals all Add filter

Name	Resource Group	Location	Subscription
shashi-vnet	shashi	Central India	Azure subscription 1
vnet-electrogo-prod	ElectroGo-Prod-RG	Australia East	Azure subscription 1

Showing 1 - 2 of 2. Display count: auto

Microsoft Azure | vnet-electrogo-prod | Subnets

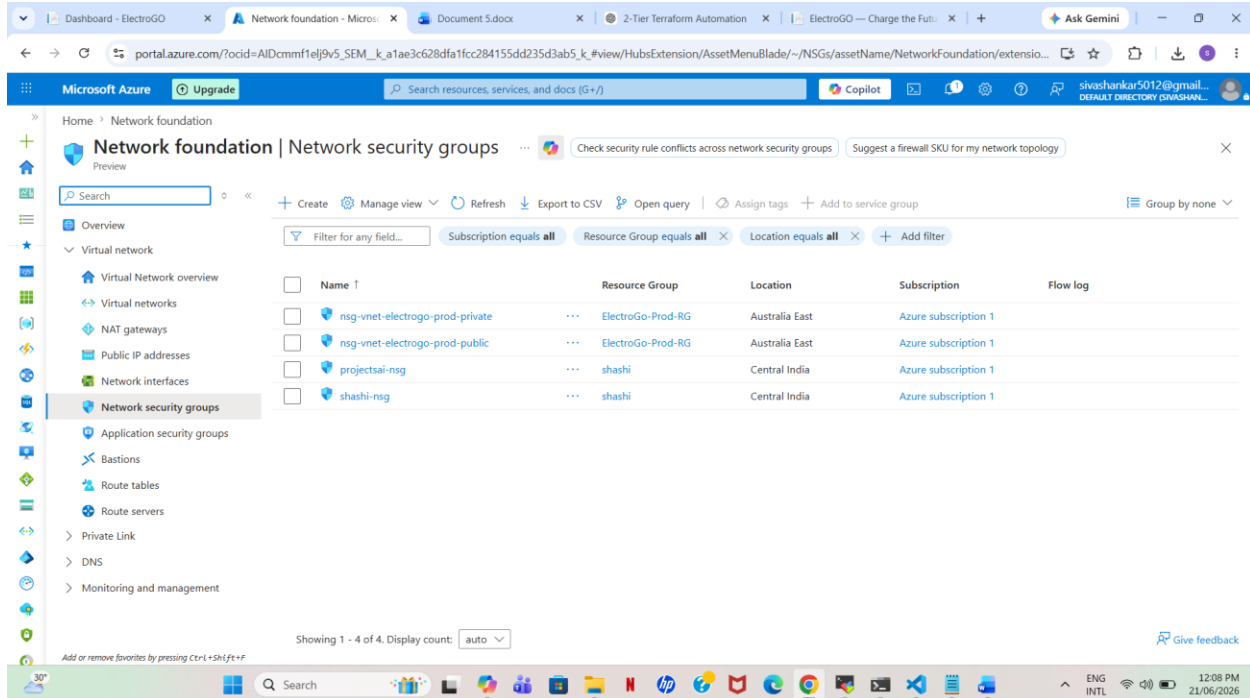
Search resources, services, and docs (G+)

Search subnets

Name	IPv4	IPv6	Available IPs	Delegated to	Security group	Route table
snet-electrogo-public	10.0.1.0/24	-	250	-	nsg-vnet-e...	-
snet-electrogo-private	10.0.2.0/24	-	251	Microsoft...	nsg-vnet-e...	-

Showing 2 subnets

NSG rules created by the network module



12. Compute Module

The **compute module** provisions the application hosting infrastructure.

Resources handled by the compute module

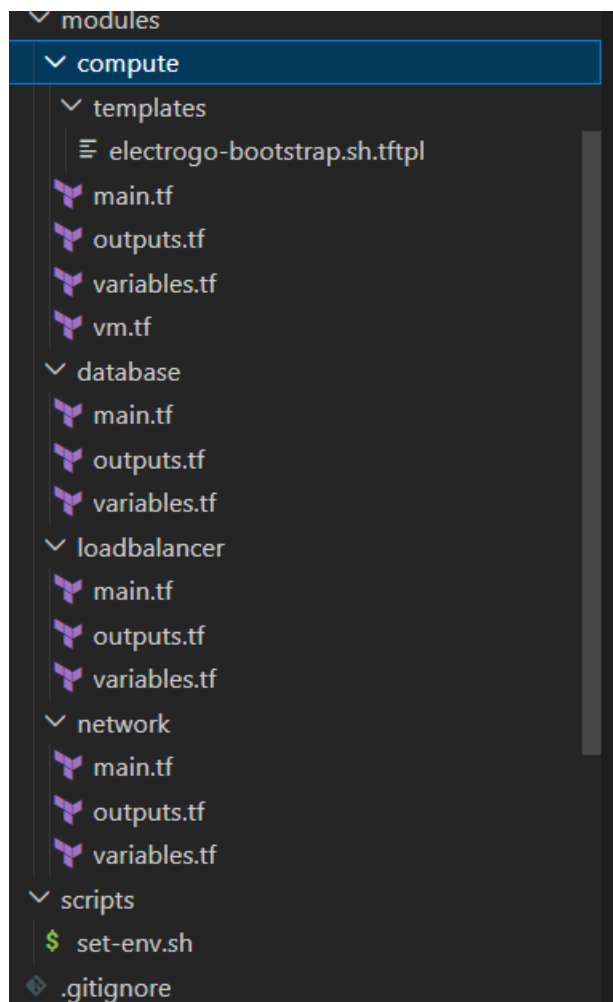
- **Network Interface (NIC)**
- **Public IP**
- **Linux Virtual Machine**

This module is responsible for creating the VM that hosts the ElectroGO web application. It also connects the VM to the required networking resources and passes provisioning data for application setup.

The compute module includes:

- `main.tf`
- `vm.tf`
- `variables.tf`
- `outputs.tf`

modules/compute folder structure



Azure VM / NIC / Public IP created by the compute module

Virtual machines

Name	Subscription	Resource Group	Private IP address	Public IP address	Location	Status	Operating system	Size
projectsai	Azure subscrip...	shashi	10.0.0.5	40.80.87.238	Central India	Running	Linux	Standard_D2
vm-electrogo-prod	Azure subscrip...	ElectroGo-Prod...	10.0.14	20.193.50.192	Australia East	Running	Linux	Standard_D2

Public IP addresses

Name	IP address	IP version	SKU	Associated to
pip-lb-electrogo-prod	20.213.38.192	IPv4	Standard	lb-electrogo-prod
pip-vm-electrogo-prod	20.193.50.192	IPv4	Standard	nic-vm-electrogo-prod
projectsai-ip	40.80.87.238	IPv4	Standard	projectsai903
shashi-ip	20.204.56.130	IPv4	Standard	shashi389

13. Templates Folder and Bootstrap Provisioning

Inside the Terraform repository, the **templates** folder contains the file:

electrogo-bootstrap.sh.tftpl

This is a **Terraform template-based shell script** that is executed during VM provisioning. It automates the server-side configuration of the Linux VM and prepares the ElectroGO application environment.

Responsibilities of the bootstrap script

- install **.NET 8**
- install **Nginx**
- install required Linux packages
- prepare application deployment directories
- pull or deploy the ElectroGO application
- configure the application with the database endpoint
- start the application and supporting services

This file is one of the most important parts of the project because it converts the raw VM into a fully configured application server.

electrogo-bootstrap.sh.tftpl inside templates folder

```
1  #!/bin/bash
2  set -xe
3  exec > /var/log/user-data.log 2>&1
4
5  # =====
6  # Terraform-injected values
7  # =====
8  APP_REPO_URL="${git_repo_url}"
9  APP_BRANCH="${git_branch}"
10 DB_HOST="${db_host}"
11 DB_PORT="${db_port}"
12 DB_NAME="${db_name}"
13 DB_USERNAME="${db_username}"
14 DB_PASSWORD="${db_password}"
15 APP_USER="${vm_admin_username}"
16
17 # =====
18 # Constants
19 # =====
20 APP_PORT="5000"
21 APP_SRC_DIR="/home/${vm_admin_username}/src"
22 APP_DIR="/home/${vm_admin_username}/app"
23
24 export HOME=/root
25 export DOTNET_CLI_HOME=/root
26 export DEBIAN_FRONTEND=noninteractive
27
28 echo "===== USERDATA START ====="
29 echo "REPO: $APP_REPO_URL"
30 echo "BRANCH: $APP_BRANCH"
31 echo "USER: $APP_USER"
32 echo "DB: $DB_USERNAME@$DB_HOST:$DB_PORT/$DB_NAME"
33
34 # =====
35 # Wait for apt lock
36 # =====
37 for i in 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20; do
```

14. Database Module

The **database module** provisions the **Azure Database for MySQL Flexible Server** used by ElectroGO.

Resources handled by the database module

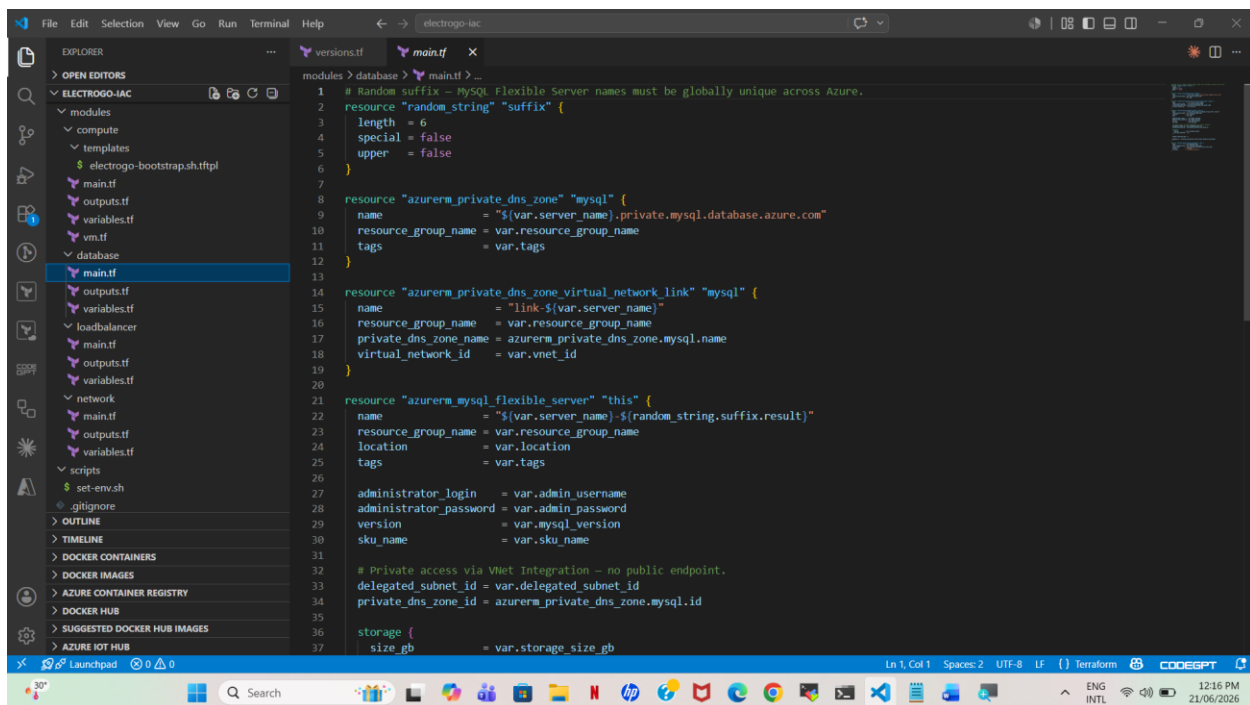
- Azure Database for MySQL Flexible Server
- related database configuration outputs

The database module contains:

- main.tf
- variables.tf
- outputs.tf

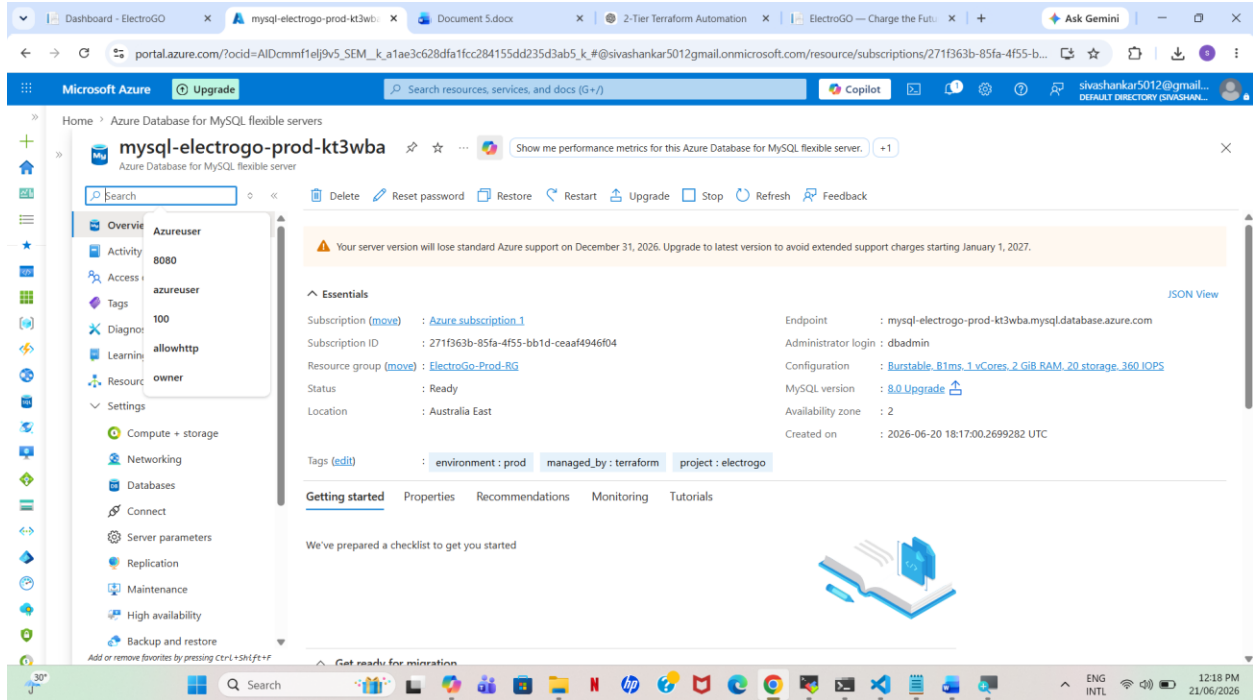
This module creates the database layer separately from the application layer and exposes outputs such as the **MySQL FQDN**, which is later used by the application deployment.

modules/database folder structure



```
1 # Random suffix - MySQL Flexible Server names must be globally unique across Azure.
2 resource "random_string" "suffix" {
3   length      = 6
4   special     = false
5   upper       = false
6 }
7
8 resource "azurerm_private_dns_zone" "mysql" {
9   name                = "${var.server_name}.private.mysql.database.azure.com"
10  resource_group_name = var.resource_group_name
11  tags                 = var.tags
12 }
13
14 resource "azurerm_private_dns_zone_virtual_network_link" "mysql" {
15   name                  = "link-${var.server_name}"
16   resource_group_name   = var.resource_group_name
17   private_dns_zone_name = azurerm_private_dns_zone.mysql.name
18   virtual_network_id     = var.vnet_id
19 }
20
21 resource "azurerm_mysql_flexible_server" "this" {
22   name                = "${var.server_name}-${random_string.suffix.result}"
23   resource_group_name = var.resource_group_name
24   location            = var.location
25   tags                = var.tags
26
27   administrator_login     = var.admin_username
28   administrator_password = var.admin_password
29   version                 = var.mysql_version
30   sku_name                = var.sku_name
31
32   # Private access via VNet Integration - no public endpoint.
33   delegated_subnet_id = var.delegated_subnet_id
34   private_dns_zone_id = azurerm_private_dns_zone.mysql.id
35
36   storage {
37     size_gb = var.storage_size_gb
38   }
39 }
```

Azure Database for MySQL Flexible Server resource created by Terraform



15. Load Balancer Module

The **loadbalancer module** provisions the **Azure Load Balancer** used in the ElectroGO deployment.

Resources handled by the load balancer module

- Azure Load Balancer
- related frontend/backend IP configuration and outputs

This module contains:

- main.tf
- variables.tf
- outputs.tf

The load balancer helps route or expose traffic for the application layer and provides an additional layer of scalability and structured traffic handling.

modules/loadbalancer folder structure

```

1 resource "azurerm_public_ip" "lb" {
2   name                = "pip-${var.lb_name}"
3   location             = var.location
4   resource_group_name = var.resource_group_name
5   allocation_method   = "Static"
6   sku                 = "Standard"
7   tags                = var.tags
8 }
9
10 resource "azurerm_lb" "this" {
11   name                = var.lb_name
12   location             = var.location
13   resource_group_name = var.resource_group_name
14   sku                 = "Standard"
15   tags                = var.tags
16 }
17
18 frontend_configuration {
19   name                = "feip-public"
20   public_ip_address_id = azurerm_public_ip.lb.id
21 }
22
23 resource "azurerm_lb_backend_address_pool" "this" {
24   loadbalancer_id = azurerm_lb.this.id
25   name            = "bep-app"
26 }
27
28 resource "azurerm_lb_probe" "http" {
29   loadbalancer_id = azurerm_lb.this.id
30   name            = "hp-http"
31   protocol        = "http"
32   port            = 80
33   request_path    = "/"
34 }
35
36 resource "azurerm_lb_rule" "http" {
37   loadbalancer_id = azurerm_lb.this.id

```

Azure Load Balancer resource overview

Home > Load balancing and content delivery > Load balancers

lb-electrogo-prod Load balancer

Is my load balancer resilient? Run diagnostic for my Load Balancer Analyze alerts for this load balancer

Search

Move Delete Refresh Give feedback

Overview

Activity log Access control (IAM) Tags Diagnose and solve problems Resource visualizer Settings

Frontend IP configuration Backend pools Health probes Load balancing rules Inbound NAT rules Outbound rules Properties Locks Monitoring

Essentials

Resource group (move) : [ElectroGo-Prod-RG](#) Location (move) : [Australia East](#) Subscription (move) : [Azure subscription 1](#) Subscription ID : 271f363b-85fa-4f55-bb1d-ceaaf4946f04 SKU : Standard Tier : Regional

Backend pool : [bep-app \(1 virtual machine\)](#) Load balancing rule : [lbr-http \(Tcp/80\)](#) Health probe : [hp-http \(Http/80\)](#) Inbound NAT rules : - Outbound rules : - Frontend IP address : -

Tags (edit) : environment : prod managed_by : terraform Show more (1)

Configure high availability and scalability for your applications

Create highly-available and scalable applications in minutes by using built-in load balancing for cloud services and virtual machines. Azure Load Balancer supports TCP/UDP-based protocols and protocols used for real-time voice and video messaging applications. [Learn more](#)

Balance IPv4 and IPv6 addresses

Native dual-stack endpoints help meet regulatory requirements and address the fast-growing number of devices in mobile and IoT. [Learn more](#)

View frontend IP configuration

Build highly reliable applications

Load Balancer improves application uptime by routing traffic to healthy nodes. [Learn more](#)

View health probes

Secure your networks

Control network traffic and protect private networks using built-in network address translation (NAT). [Learn more](#)

View inbound NAT rules

View backend pools View load balancing rules

16. Scripts Folder

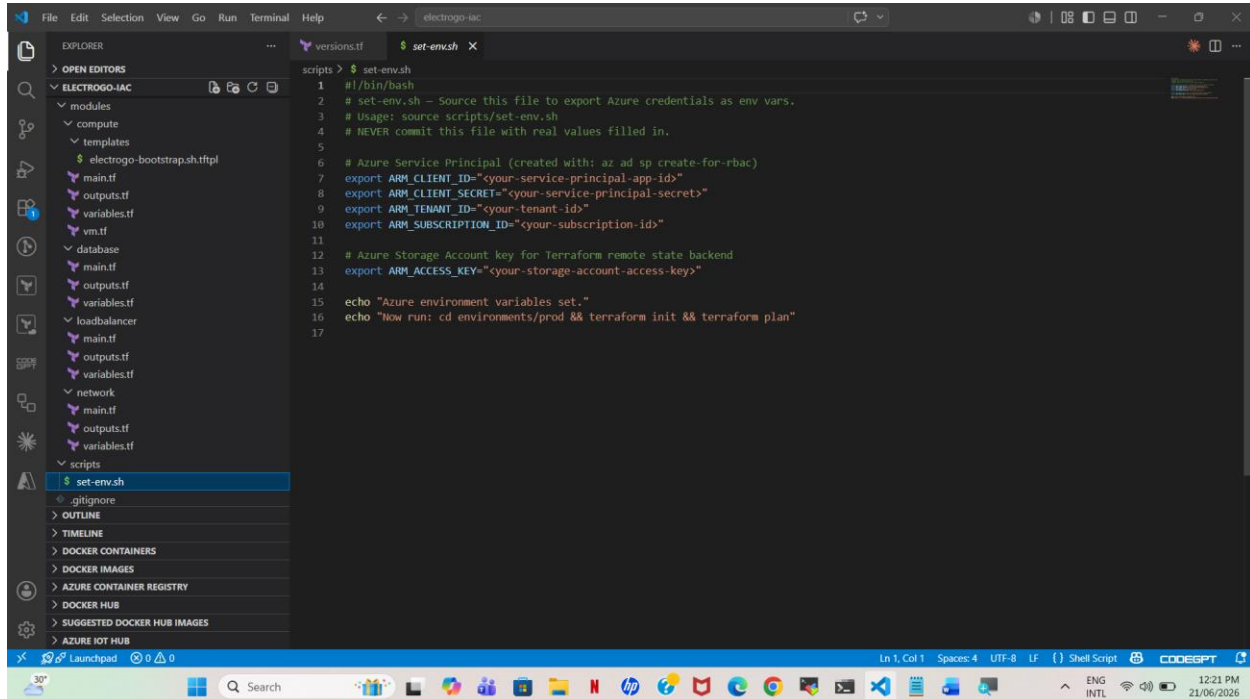
The repository also contains a **scripts** folder with:

set-env.sh

This script is used to **export ARM environment variables / Azure credentials** required for Terraform authentication and execution.

This helps standardize environment configuration before running Terraform commands.

scripts/set-env.sh



```
1 #!/bin/bash
2 # set-env.sh -- Source this file to export Azure credentials as env vars.
3 # Usage: source scripts/set-env.sh
4 # NEVER commit this file with real values filled in.
5
6 # Azure Service Principal (created with: az ad sp create-for-rbac)
7 export ARM_CLIENT_ID="your-service-principal-app-id"
8 export ARM_CLIENT_SECRET="your-service-principal-secret"
9 export ARM_TENANT_ID="your-tenant-id"
10 export ARM_SUBSCRIPTION_ID="your-subscription-ids"
11
12 # Azure Storage Account key for Terraform remote state backend
13 export ARM_ACCESS_KEY="your-storage-account-access-key"
14
15 echo "Azure environment variables set."
16 echo "Now run: cd environments/prod && terraform init && terraform plan"
17
```

17. Terraform Outputs Used in the Project

The Terraform project exposes important deployment outputs through `outputs.tf`. These outputs help verify the deployment and provide the values required to access the deployed infrastructure.

Key outputs in this project

- **Virtual Machine IP**
- **Load Balancer IP**
- **MySQL FQDN**

These outputs are especially useful because:

- the **VM IP / LB IP** is used to access the ElectroGO web application
- the **MySQL FQDN** is used for application database connectivity

- they provide immediate visibility into the deployed resources after terraform apply

Terraform output values after deployment

```
module.loadbalancer.azure_rm_lb_backend_address_pool.this: Creating...
module.loadbalancer.azure_rm_lb_probe.http: Still creating... [00m10s elapsed]
module.loadbalancer.azure_rm_lb_backend_address_pool.this: Still creating... [00m10s elapsed]
module.loadbalancer.azure_rm_lb_probe.http: Creation complete after 12s [id=/subscriptions/271f363b-85fa-4f55-bb1d-ceaaf4946f04/resourceGroups/ElectroGo-Prod-RG/providers/Microsoft.Network/loadBalancers/lb-electrogo-prod/probes/hp-http]
module.loadbalancer.azure_rm_lb_backend_address_pool.this: Creation complete after 15s [id=/subscriptions/271f363b-85fa-4f55-bb1d-ceaaf4946f04/resourceGroups/ElectroGo-Prod-RG/providers/Microsoft.Network/loadBalancers/lb-electrogo-prod/backendAddressPools/bep-app]
module.loadbalancer.azure_rm_network_interface_backend_address_pool_association.vm: Creating...
module.loadbalancer.azure_rm_lb_rule.http: Creating...
module.loadbalancer.azure_rm_network_interface_backend_address_pool_association.vm: Creation complete after 3s [id=/subscriptions/271f363b-85fa-4f55-bb1d-ceaaf4946f04/resourceGroups/ElectroGo-Prod-RG/providers/Microsoft.Network/networkInterfaces/nic-vm-electrogo-prod/ipConfigurations/internal/subscriptions/271f363b-85fa-4f55-bb1d-ceaaf4946f04/resourceGroups/ElectroGo-Prod-RG/providers/Microsoft.Network/loadBalancers/lb-electrogo-prod/backendAddressPools/bep-app]
module.loadbalancer.azure_rm_lb_rule.http: Creation complete after 6s [id=/subscriptions/271f363b-85fa-4f55-bb1d-ceaaf4946f04/resourceGroups/ElectroGo-Prod-RG/providers/Microsoft.Network/loadBalancers/lb-electrogo-prod/loadBalancingRules/lbr-http]
Releasing state lock. This may take a few moments...

Apply complete! Resources: 22 added, 0 changed, 0 destroyed.

Outputs:
load_balancer_public_ip = "20.58.179.54"
mysql_fqdn = "mysql-electrogo-prod-on2pks.mysql.database.azure.com"
mysql_server_name = "mysql-electrogo-prod-on2pks"
resource_group_name = "ElectroGo-Prod-RG"
vm_principal_id = "ada23929-52d1-4fc3-8d36-7e7a6e2dfcfd"
vm_public_ip = "13.75.223.136"
PS C:\Users\Siva Shankar\OneDrive\Desktop\electrogo-iac> |
```

18. Deployment Workflow

- The deployment process is fully automated through Terraform and the ElectroGO bootstrap script. The workflow is as follows:
- Terraform is executed from the root `electrogo-iac` directory.
- Terraform initializes the Azure provider and remote backend.
- Terraform reads the state file from Azure Blob Storage.
- Terraform processes the root configuration files and loads the required modules.
- The **network module** provisions the VNet, subnets, and NSGs.
- The **database module** provisions Azure Database for MySQL Flexible Server.
- The **compute module** provisions the Linux VM, NIC, and public IP.
- The **loadbalancer module** provisions the Azure Load Balancer.
- During VM provisioning, the **electrogo-bootstrap.sh.tftpl** script runs automatically.
- The script installs .NET 8, Nginx, and application dependencies.
- The ElectroGO application is deployed and configured on the VM.
- The generated MySQL FQDN is inserted into the application configuration.
- The application is started and exposed through the configured network path.

terraform init execution

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\Siva Shankar\OneDrive\Desktop\electrogo-iac> terraform init
Initializing the backend...
Initializing modules...
Initializing provider plugins...
- Reusing previous version of hashicorp/azurerm from the dependency lock file
- Reusing previous version of hashicorp/random from the dependency lock file
- Using previously-installed hashicorp/random v3.9.0
- Using previously-installed hashicorp/azurerm v3.117.1

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\Users\Siva Shankar\OneDrive\Desktop\electrogo-iac> |
```

terraform plan validate

```
If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\Users\Siva Shankar\OneDrive\Desktop\electrogo-iac> terraform validate
Success! The configuration is valid.

PS C:\Users\Siva Shankar\OneDrive\Desktop\electrogo-iac> |
```

terraform plan

```
PS C:\Users\Siva Shankar\OneDrive\Desktop\electrogo-iac> terraform plan
Acquiring state lock. This may take a few moments...
azurerm_resource_group.main: Refreshing state... [id=/subscriptions/271f363b-85fa-4f55-bb1d-ceaaf4946f04/resourceGroups/ElectroGo-Prod-RG]
module.network.azure_rm_virtual_network.this: Refreshing state... [id=/subscriptions/271f363b-85fa-4f55-bb1d-ceaaf4946f04/resourceGroups/ElectroGo-Prod-RG/providers/Microsoft.Network/virtualNetworks/vnet-electrogo-prod]
module.network.azure_rm_network_security_group.public: Refreshing state... [id=/subscriptions/271f363b-85fa-4f55-bb1d-ceaaf4946f04/resourceGroups/ElectroGo-Prod-RG/providers/Microsoft.Network/networkSecurityGroups/nsg-vnet-electrogo-prod-public]
module.network.azure_rm_network_security_group.private: Refreshing state... [id=/subscriptions/271f363b-85fa-4f55-bb1d-ceaaf4946f04/resourceGroups/ElectroGo-Prod-RG/providers/Microsoft.Network/networkSecurityGroups/nsg-vnet-electrogo-prod-private]
module.network.azure_rm_subnet.public: Refreshing state... [id=/subscriptions/271f363b-85fa-4f55-bb1d-ceaaf4946f04/resourceGroups/ElectroGo-Prod-RG/providers/Microsoft.Network/virtualNetworks/vnet-electrogo-prod/subnets/snet-electrogo-public]
module.network.azure_rm_subnet.private: Refreshing state... [id=/subscriptions/271f363b-85fa-4f55-bb1d-ceaaf4946f04/resourceGroups/ElectroGo-Prod-RG/providers/Microsoft.Network/virtualNetworks/vnet-electrogo-prod/subnets/snet-electrogo-private]
module.network.azure_rm_subnet_network_security_group_association.public: Refreshing state... [id=/subscriptions/271f363b-85fa-4f55-bb1d-ceaaf4946f04/resourceGroups/ElectroGo-Prod-RG/providers/Microsoft.Network/virtualNetworks/vnet-electrogo-prod/subnets/snet-electrogo-public]
module.network.azure_rm_subnet_network_security_group_association.private: Refreshing state... [id=/subscriptions/271f363b-85fa-4f55-bb1d-ceaaf4946f04/resourceGroups/ElectroGo-Prod-RG/providers/Microsoft.Network/virtualNetworks/vnet-electrogo-prod/subnets/snet-electrogo-private]
module.database.azure_rm_private_dns_zone.mysql: Refreshing state... [id=/subscriptions/271f363b-85fa-4f55-bb1d-ceaaf4946f04/resourceGroups/ElectroGo-Prod-RG/providers/Microsoft.Network/privateDnsZones/mysql-electrogo-prod.private.mysql.data.base.azure.com]
module.database.random_string.suffix: Refreshing state... [id=kt3wba]
module.database.azure_rm_private_dns_zone_virtual_network_link.mysql: Refreshing state... [id=/subscriptions/271f363b-85fa-4f55-bb1d-ceaaf4946f04/resourceGroups/ElectroGo-Prod-RG/providers/Microsoft.Network/privateDnsZoneVirtualNetworkLinks/mysql-electrogo-prod.private.mysql.vnet-link]
```

terraform apply output

```
PS C:\Users\Siva Shankar\OneDrive\Desktop\electrogo-iac> terraform apply -auto-approve
Acquiring state lock. This may take a few moments...
azurerm_resource_group.main: Refreshing state... [id=/subscriptions/271f363b-85fa-4f55-bb1d-ceaaf4946f04/resourceGroups/ElectroGo-Prod-RG]
module.network.azure_rm_virtual_network.this: Refreshing state... [id=/subscriptions/271f363b-85fa-4f55-bb1d-ceaaf4946f04/resourceGroups/ElectroGo-Prod-RG/providers/Microsoft.Network/virtualNetworks/vnet-electrogo-prod]
module.network.azure_rm_network_security_group.public: Refreshing state... [id=/subscriptions/271f363b-85fa-4f55-bb1d-ceaaf4946f04/resourceGroups/ElectroGo-Prod-RG/providers/Microsoft.Network/networkSecurityGroups/nsg-vnet-electrogo-prod-public]
module.network.azure_rm_network_security_group.private: Refreshing state... [id=/subscriptions/271f363b-85fa-4f55-bb1d-ceaaf4946f04/resourceGroups/ElectroGo-Prod-RG/providers/Microsoft.Network/networkSecurityGroups/nsg-vnet-electrogo-prod-private]
module.network.azure_rm_subnet.public: Refreshing state... [id=/subscriptions/271f363b-85fa-4f55-bb1d-ceaaf4946f04/resourceGroups/ElectroGo-Prod-RG/providers/Microsoft.Network/virtualNetworks/vnet-electrogo-prod/subnets/snet-electrogo-public]
module.network.azure_rm_subnet.private: Refreshing state... [id=/subscriptions/271f363b-85fa-4f55-bb1d-ceaaf4946f04/resourceGroups/ElectroGo-Prod-RG/providers/Microsoft.Network/virtualNetworks/vnet-electrogo-prod/subnets/snet-electrogo-private]
module.network.azure_rm_subnet_network_security_group_association.private: Refreshing state... [id=/subscriptions/271f363b-85fa-4f55-bb1d-ceaaf4946f04/resourceGroups/ElectroGo-Prod-RG/providers/Microsoft.Network/virtualNetworks/vnet-electrogo-prod/subnets/snet-electrogo-private]
module.network.azure_rm_subnet_network_security_group_association.public: Refreshing state... [id=/subscriptions/271f363b-85fa-4f55-bb1d-ceaaf4946f04/resourceGroups/ElectroGo-Prod-RG/providers/Microsoft.Network/virtualNetworks/vnet-electrogo-prod/subnets/snet-electrogo-public]
module.database.azure_rm_private_dns_zone.mysql: Refreshing state... [id=/subscriptions/271f363b-85fa-4f55-bb1d-ceaaf4946f04/resourceGroups/ElectroGo-Prod-RG/providers/Microsoft.Network/privateDnsZones/mysql-electrogo-prod.private.mysql.data.base.azure.com]
module.database.random_string.suffix: Refreshing state... [id=kt3wba]
module.database.azure_rm_private_dns_zone_virtual_network_link.mysql: Refreshing state... [id=/subscriptions/271f363b-85fa-4f55-bb1d-ceaaf4946f04/resourceGroups/ElectroGo-Prod-RG/providers/Microsoft.Network/privateDnsZoneVirtualNetworkLinks/mysql-electrogo-prod.private.mysql.vnet-link]
```

19. Automated Application Provisioning

The ElectroGO application deployment is automated through the **bootstrap template** attached to the VM provisioning process.

Provisioning activities performed on the VM

- install required Linux packages
- install **.NET 8**
- install **Nginx**
- prepare the deployment directory
- pull application source code
- inject database endpoint into configuration
- publish / start the application
- expose it through the server configuration

This ensures that after Terraform completes, the VM is not just created but also configured as a working ElectroGO application server.

Bootstrap script section showing package installation and application setup

```
#!/bin/bash
set -xe
exec > /var/log/user-data.log 2>&1

# =====
# Terraform-injected values
# =====
APP_REPO_URL="${git_repo_url}"
APP_BRANCH="${git_branch}"
DB_HOST="${db_host}"
DB_PORT="${db_port}"
DB_NAME="${db_name}"
DB_USERNAME="${db_username}"
DB_PASSWORD="${db_password}"
APP_USER="${vm_admin_username}"

# =====
# Constants
# =====
APP_PORT="5000"
APP_SRC_DIR="/home/${vm_admin_username}/src"
APP_DIR="/home/${vm_admin_username}/app"

export HOME=/root
export DOTNET_CLI_HOME=/root
export DEBIAN_FRONTEND=noninteractive

echo "==== USERDATA START ====="
echo "REPO: $APP_REPO_URL"
echo "BRANCH: $APP_BRANCH"
echo "USER: $APP_USER"
echo "DB: $DB_USERNAME@$DB_HOST:$DB_PORT/$DB_NAME"

# =====
# Wait for apt lock
# =====
for i in 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20; do
```

20. Database Connectivity and MySQL FQDN Injection

The ElectroGO application requires connectivity to the Azure Database for MySQL Flexible Server. Since the database server hostname is generated dynamically during deployment, the application cannot use a hardcoded value.

To solve this, the Terraform deployment and bootstrap configuration pass the **MySQL FQDN** to the application during provisioning. This value is then inserted into the application's configuration so that the ElectroGO web application can connect to the correct Azure database instance.

Values typically injected into the application

- MySQL FQDN / database endpoint
- database name
- database username
- database password
- database port

This approach ensures fully automated integration between the application tier and database tier.

Terraform output showing MySQL FQDN

 **Insert Screenshot Here**

Screenshot 29: Bootstrap / configuration section showing database endpoint injection

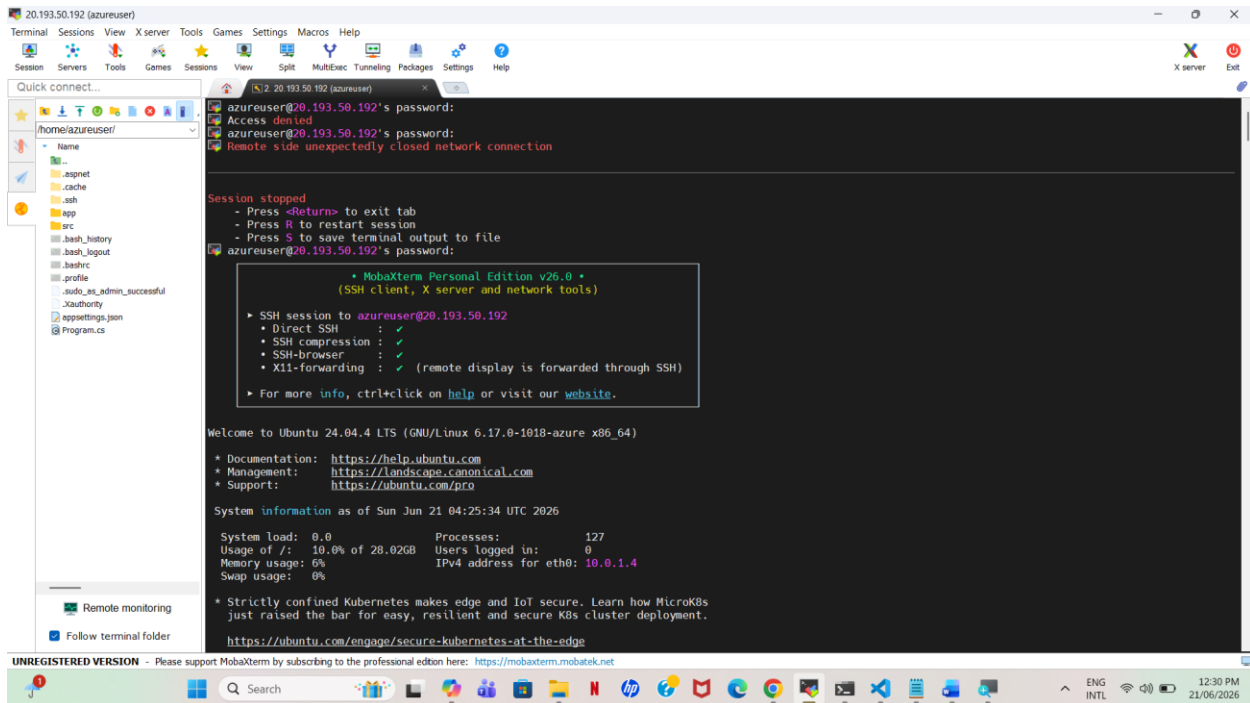
21. Validation and Testing

The deployment was validated using the following checks:

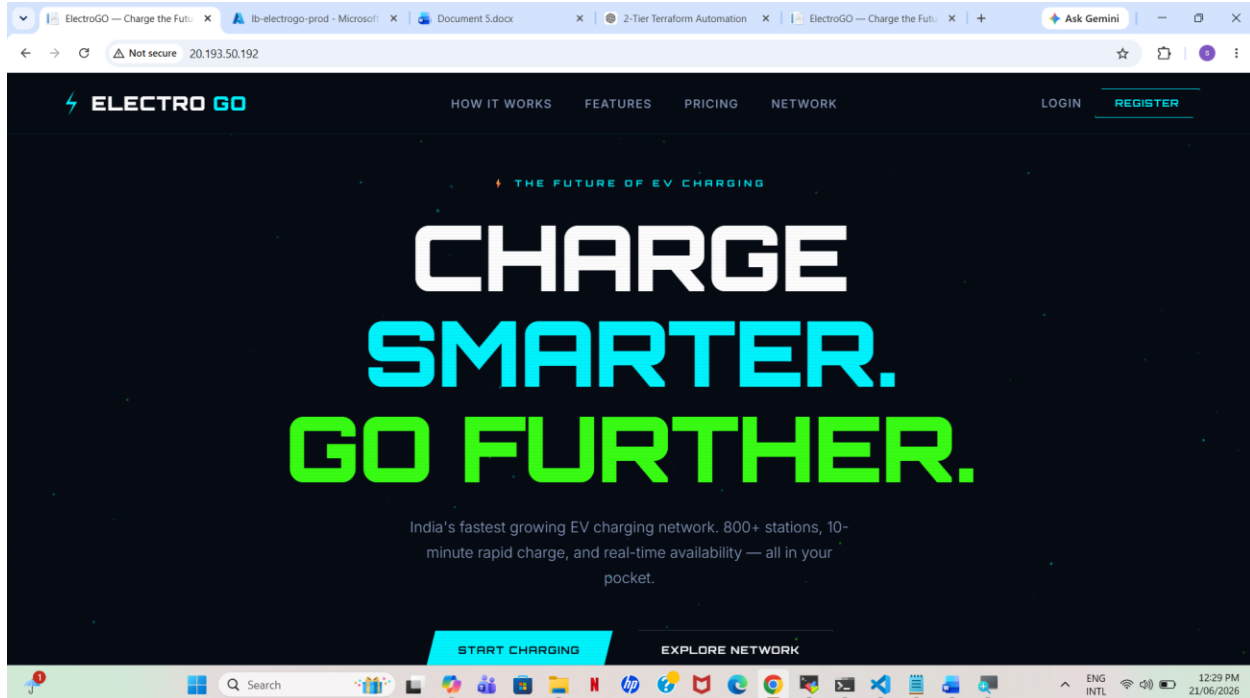
1. Terraform completed successfully without errors.
2. The Azure portal showed all expected resources:
 - a. VNet
 - b. NSG
 - c. Linux VM
 - d. Public IP
 - e. Load Balancer
 - f. MySQL Flexible Server
3. The VM was accessible through SSH.

4. The ElectroGO application server was configured automatically.
5. The application was reachable through the VM IP or Load Balancer IP.
6. The application was connected successfully to the Azure MySQL database.
7. Terraform outputs displayed the expected IP addresses and MySQL FQDN.

VM access / SSH verification



ElectroGO application running in browser



ElectroGo Database

```

Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 29
Server version: 8.0.44-azure Source distribution

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> USE appdb;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> DESCRIBE users;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| Id     | int  | NO   | PRI | NULL    | auto_increment |
| Username | longtext | NO   |     | NULL    |                |
| Email  | longtext | NO   |     | NULL    |                |
| PasswordHash | longtext | NO   |     | NULL    |                |
| FullName | longtext | NO   |     | NULL    |                |
| Phone  | longtext | NO   |     | NULL    |                |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.02 sec)

mysql> SELECT * FROM users;
+-----+-----+-----+-----+-----+-----+
| Id | Username | Email | PasswordHash | FullName | Phone |
+-----+-----+-----+-----+-----+-----+
| 1 | sivashankar | sivashankar@gmail.com | A0AAAAIAAYagAAAAEPbyIZoSV6/e51+BxhfEPrygVr1JyZ1Vk8yLEajJMafEY/7DKrBw4xNDLfeKVfd1Kw== | sivashankar | 9676568928 |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> exit
bye

```

22. Benefits of the Implemented Terraform Design

This project provides several advantages:

- **modular Terraform structure** improves code organization and reuse
- **network, compute, database, and load balancer layers are separated clearly**

- **remote backend** improves Terraform state management
- **bootstrap provisioning** automates application setup after infrastructure creation
- **dynamic MySQL endpoint injection** removes manual configuration work
- **repeatable Azure deployment workflow** reduces human error
- **.NET application deployment is integrated directly into infrastructure provisioning**
- **load balancer support** improves traffic handling design

23. Conclusion

This project successfully demonstrates the deployment of the **ElectroGO ASP.NET Core (.NET 8) web application** on **Microsoft Azure** using a **modular Terraform architecture**. The deployment provisions the required network, compute, database, and load balancing resources, and automatically configures the Linux virtual machine to host the application.

The use of separate Terraform modules for **network, compute, database, and loadbalancer** makes the infrastructure easier to manage and extend. The **ElectroGO bootstrap template** automates the installation of .NET 8, Nginx, and application setup on the VM, while the **MySQL FQDN** is dynamically passed to the application to establish database connectivity.

Overall, this project provides a practical implementation of **Infrastructure as Code, Azure cloud deployment, and automated .NET application provisioning** in a 2-tier architecture.

